Pamantasan Ng Lungsod Ng Maynila

Intramuros, Manila

College of Information Systems and Technology Management

Department of Computer Science

May 4, 2025

This is to certify that the manuscript entitled Smart Budget Tracker: Python-Based Income & Expense Manager, by the following students:

Avila, Julien Jamile P.

Dela Cruz, Lorejane M.

Guevarra, Gabrielle Ruth V.

Jandoc, Russel Paolo C.

Martin, Scheanz Y.

Sebastian, Jan Zyanne B.

In compliance, the project presentation is found to be complete and checked by the undersigned.

Prof. Marilou B. Mangrobang

SMART BUDGET TRACKER: PYTHON-BASED INCOME & EXPENSE MANAGER

A Project Presentation

Presented to

The Faculty of College of Information Systems and Technology Management

COMPUTER SCIENCE DEPARTMENT

In Partial Fulfillment

Of the Requirements for the Degree in

Bachelor of Science in Computer Science

By:

Avila, Julien Jamile P.

Dela Cruz, Lorejane M.

Guevarra, Gabrielle Ruth V.

Jandoc, Russel Paolo C.

Martin, Scheanz Y.

Sebastian, Jan Zyanne B.

May 4, 2025

ACKNOWLEDGEMENT

Through the journey of this study, we, researchers, faced multiple lessons, challenges, and unexpected turns that pushed us for growth. As we reach the conclusion , we would like to take a moment to recognize those people who helped us reach this far.

Above all, we are thankful for God, our creator, who showed us the way for success and guided us through every step that we, researchers, take. We are overwhelmed by the lessons and success that he gave us throughout this study.

We, researchers, show our gratitude and appreciation to our panelist, who gave us some time to consult and be critical towards our study. We appreciate their willingness to guide us in this study.

To our most gorgeous research advisor, Marilou B. Mangrobang, who guided us not only in this research but also in our Python learning journey. We appreciate her time, knowledge, and efforts that played a huge role in completing this study.

To our families and friends, who are the backbone of our daily lives and gave us energy and encouragement, we extend our deepest appreciation.

ABSTRACT

In an era when financial self-control is essential, many people continue to manage their spending manually using calculators and manual records, which are frequently prone to mistakes, inefficiencies, and inconsistencies. The purpose of this study is to address common financial management problems that these users encounter, such as the tendency to ignore certain expenses and their lack of motivation to regularly monitor their accounts. Python was used to create an intelligent budget tracking system in order to address these issues.

The system is intended to provide visual financial insights, category budget management, and real-time spending tracking. It presents elements that promote user interaction and pay attention to expenses that are often ignored. This technology gives people from those with salaries to freelancers and students, more financial control by automating the budgeting process and showing data in a way that can be understood. In the end, the system encourages better spending practices and makes it easier and more aware for individuals to work toward financial stability.

TABLE OF CONTENTS

PAGE

PRELIMINARIES

TITLE PAGE ---------- 1

ACKNOWLEDGEMENT ---------- 3

ABSTRACTS ---------- 4

LIST OF TABLES ---------- 5

LIST OF FIGURES ---------- 6

CHAPTER 1 INTRODUCTION

PROJECT RATIONALE ---------- 7

STATEMENT OF THE PROBLEM ---------- 7

OBJECTIVE OF THE STUDY ---------- 7

SCOPE AND LIMITATIONS ---------- 9

SIGNIFICANCE OF THE STUDY ---------- 8

DEFINITION OF TERMS ---------- 10

CHAPTER 2 CONCEPTUAL FRAMEWORK

FRAMEWORK OF THE SYSTEM ---------- 12

FLOWCHART ---------- 14

CHAPTER 3 TECHNICAL BACKGROUND

HARDWARE SPECIFICATIONS - 16

SOFTWARE SPECIFICATIONS - 16

APPENDICES

SOURCE CODES - 16

SYSTEM LAYOUT/DESIGN - 33

CURRICULUM VITAE - 34

LIST OF FIGURES

FIGURE **PAGE**

1 FRAMEWORK OF THE SYSTEM - 12

2 FLOWCHART - 14

CHAPTER 1
INTRODUCTION

STATEMENT OF THE PROBLEM

1. What are some financial management issues that individuals who use manual systems like spreadsheets and paper logs face?
2. What prevents users from consistently tracking their expenses?
3. Which particular expenses are the most overlooked by individuals?

OBJECTIVE OF THE STUDY

1. To develop a financial tracking system that addresses the problem of manual methods such as spreadsheets and paper logs.
2. To create features that motivate consistent money tracking and improve user engagement.
3. To implement a feature that can create categories that highlight commonly overlooked expenses.

PROJECT RATIONALE

Effective financial control is important in today's rapidly changing environment, yet many people find it challenging to keep track of their money, expenses, and savings. Traditional techniques, such as using calculators to track or manually budgeting, can be challenging and prone to mistakes. Due to the growing dependence on digital solutions, an approach to managing more automated and efficient money is needed.

This smart budget tracker aims to provide users with a smart, user-friendly, and automated tool to monitor their financial transactions. Built using Python, the system will offer

key features such as real-time expense tracking, budget categorization, data visualization, and financial insights to help users make informed spending decisions.

This solution will improve users' financial knowledge and discipline by using Python's data handling and visualization capabilities. The system aims to enable people to take control of their money, cut back on wasteful spending, and efficiently and easily aim toward financial stability.

SIGNIFICANCE OF THE STUDY

This system project aims to track and monitor the finances of individuals, or the beneficiaries, who struggle to keep track of their monthly spending and remaining balance due to unorganized financial records or frequent expenses. The system was developed to provide a clear overview of where their money goes and how much balance they have left for the month. The following beneficiaries will benefit from this system:

1. **People with Income** – Individuals who receive a regular salary or earnings and need a system to monitor their expenses and ensure they are aware of their remaining balance.
2. **Freelancers and Gig Workers** – Individuals with inconsistent income who need a tool to track their spending and available funds throughout the month.
3. **Students and Young Professionals** – Individuals managing personal finances for the first time who need a system to help them keep track of daily expenses and prevent overspending

SCOPE AND LIMITATIONS

The study provides insights into the budget tracking system, which specifically monitors the user's expenses based on their monthly income. It addresses the problem of inconvenient expense management, such as unorganized finances and losing track of essential payments. The system offers an efficient way for users to consistently be updated on their monthly expenses, allowing for more convenient financial planning. Specifically, it outlines the system's features and the limitations encountered during its development.

Features:

- Income-Based Budget Tracking - The system uses the user's monthly income and reported expenses to create a budget.
- Expense Categories - As the user enters expenses, the system classifies them and assigns a payment amount.
- Payee Details - The user chooses the individual, business, or organization that will receive the payment.
- Date of Payment - The user has the option to designate the date of payment.
- Payment Method - The user chooses the payment method that will be applied to the transaction.
- Expense Description - You can include further information about the expense or the payee.

Limitation:

- There are no personal accounts (Log in/out) due to researchers' insufficient knowledge of databases
- The system does not support full budgeting

- The system may have limited security.
- The system does not provide external software integration.
- Users are required to execute manual inputs.

DEFINITION OF TERMS

Date of Payment

The specific date on which a financial transaction is recorded or scheduled to occur.

Expense Categories

Classifications used to organize expenditures into distinct groups based on their nature, such as utilities, food, or entertainment.

Expense Description

Additional information provided to clarify the purpose or nature of a financial transaction.

Financial Tracking System

A technological framework designed to facilitate the monitoring and management of financial transactions, including income and expenses, to enhance financial oversight.

Limited Budgeting Support

The system's inability to provide comprehensive budgeting functionalities beyond basic expense tracking.

Manual Inputs

The necessity for users to manually enter financial data into the system, as it does not support automated data collection.

Manual Methods

Traditional techniques for tracking financial data, such as using spreadsheets or paper-based logs, which can be labor-intensive and susceptible to errors.

No External Software Integration

The system's inability to interface with other financial software or applications, limiting its interoperability.

No Personal Accounts

The absence of a user authentication system, such as login and logout features, due to limitations in database management capabilities.

Payee Details

Information pertaining to the recipient of a payment, including their identity and relevant contact information.

Payment Method

The medium through which a transaction is completed, such as cash, credit card, or electronic transfer.

Security Limitations

The system's restricted capacity to ensure the security of user data due to a lack of advanced security protocols.

Users

Individuals utilizing the financial tracking system, encompass a diverse range of demographics, including those with fixed incomes, freelancers, gig workers, students, and young professionals.

Income-Based Budget Tracking

A system feature that utilizes the user's reported monthly income to manage and allocate expenses effectively.

CHAPTER 2
CONCEPTUAL FRAMEWORK

FRAMEWORK OF THE SYSTEM

INPUT	PROCESS	OUTPUT
1. The user inputs their monthly income. 2. The user logs all their daily expenses by entering the exact amount spent. 3. The user selects an expense category from a predefined list. 4. The user may add a short description or note for each expense. 5. The user designates the date of payment for each expense.	1. The system will analyze the monthly income that the user inputs. 2. The system will categorize the expenses based on the user's input on predefined categories. 3. The system will calculate the remaining balance of the user per manual log. 4. The system will use the payment date set by the user to process the expense and update the monthly balance. 5. The system adjusts the balance dynamically based on recurring expenses. 6. The system flags overspending when expenses in a specific category exceed the set budget.	1. The system displays the real-time remaining balance after each input. 2. The system generates a summary of expenses, grouped by category. 3. The system provides a table for spending patterns over time.

Figure 1: Framework of the Proposed Money-Tracking System

Figure 1 shows the framework of the proposed financial tracking system, outlining how user data is managed across different stages. In the input phase, users begin by entering their monthly income, followed by daily expense entries. Each expense is recorded with the amount spent, a selected category from a predefined list, and optional descriptive notes. Every

transaction is tagged with a payment date, enabling the system to factor in time-based elements for more accurate financial tracking.

In the processing phase, the system organizes and analyzes the input data. It categorizes expenses, calculates the real-time remaining balance by subtracting total expenses from income, and schedules deductions based on specified payment dates. To help users stay financially disciplined, the system features an alert mechanism that detects and notifies users of overspending in specific categories, encouraging adherence to their budgets.

During the output phase, the system presents the analyzed information in a clear and accessible format. Users are shown an up-to-date display of their available balance, along with summarized spending reports categorized for clarity. To enhance understanding, a table highlights spending patterns over time. This comprehensive feedback system empowers users to gain insights into their financial habits and make more informed decisions in managing their finances.

FLOWCHART

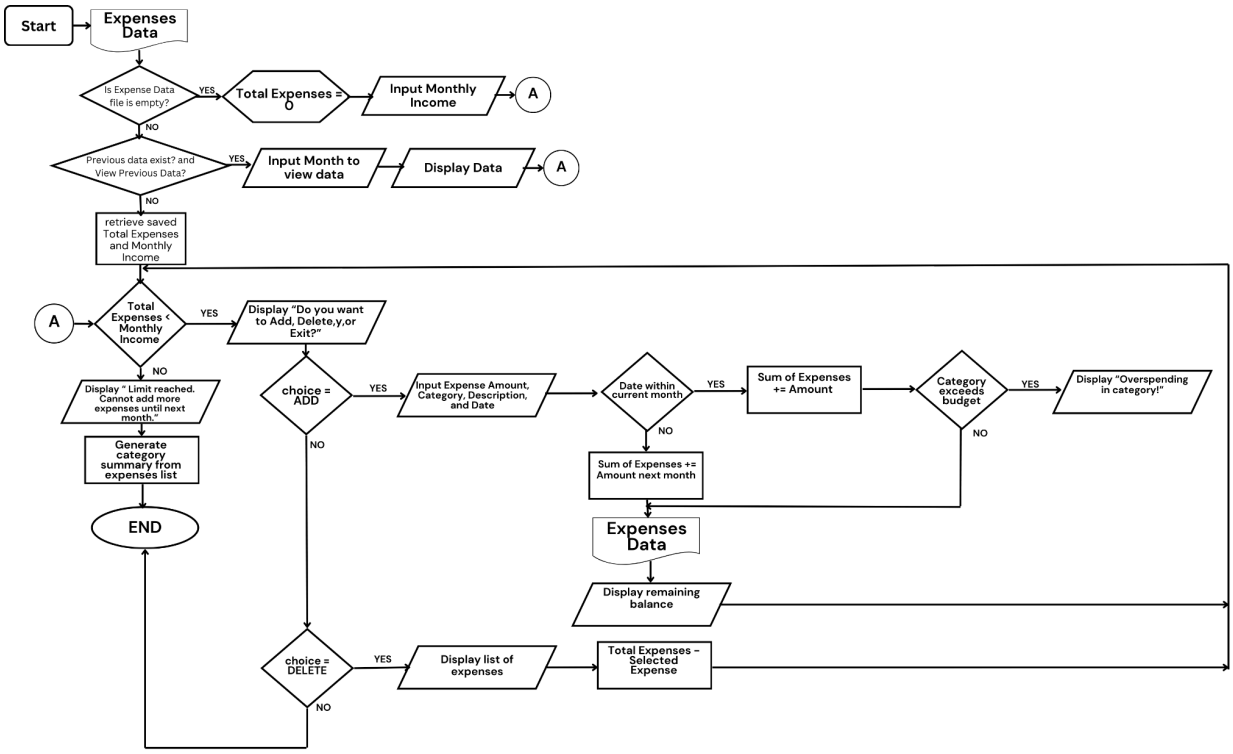


Figure 2: Flowchart of Money-Tracking System

The flowchart presents a monthly expense tracking system that begins by checking whether the data file used to store expense information is empty. If it is empty, the system initializes total expenses to zero and prompts the user to input their monthly income. If the file contains data, the system loads it back to the table for consistent updating of the expenses of the user, despite shutting down the program. Additionally, the user has an option to check data from previous month/s if they used the program more than once from those timeframes. If the user agrees, they are prompted to input a month, and the system displays the corresponding financial records.

The system keeps checking to see if total expenses have hit the monthly income limit after it has been set up or data has been retrieved. Otherwise, the user is presented with the option to add, delete, or exit.

The user must enter the amount, category, description, and date when they decide to add an expense. After that, the system determines if the expense date is within the current month. If so, the sum is added to the current month's total expenses; if not, it is put away for the following month. The system shows the remaining balance after each addition, and a warning notice indicating "Overspending in category!" appears if any expense category goes beyond budget.

The system shows a list of current expenses if the user chooses to delete. After that, the user can select one entry to delete, which changes the remaining balance and lowers the total expenses appropriately.

The system stops adding expenses and shows the notification "Limit reached" if the overall spending exceeds the monthly revenue. No further expenses can be added until the following month. The system resets the total expenses at the conclusion of each month and the start of a new one, allowing for fresh entries for the subsequent cycle. Finally, before concluding the session, the system creates a summary of all expenses grouped appropriately.

CHAPTER 3
TECHNICAL BACKGROUND

HARDWARE SPECIFICATIONS

Processor : Intel Core i5-4210M CPU, 2.60GHz

Installed Memory (RAM): 12.0GB minimum, 16.0GB recommended

System Type: 64-bit operating system

SOFTWARE SPECIFICATIONS

Operating System: Windows

Python

Visual Studio Code

Tkinter Widgets

JSON

Datetime

APPENDICES

SOURCE CODES

(main.py)

```
import tkinter as tk
from tkinter import ttk, messagebox
from datetime import datetime
import datahandling as file
import datehandling as datehandle
import errorhandling as error
import logic as logic
import datetime

root = tk.Tk()
root.title("Smart Budget Tracker")
root.geometry("900x600")
root.configure(bg='#f0f2f5')
```

```
background_color = "#f0f2f5"
primary_color = "#2d4059"
accent_color = "#5cdb95"
light_accent = "#edf5e1"
base_font = ('Segoe UI', 13)
header_font = ('Segoe UI', 18, 'bold')

current_MonthYear = datehandle.passDate()

if file.isFileEmpty() == True:
    monthly_income = 0.0
    remaining_balance = 0.0
    total_expenses = 0.0
    expenses = {}
    category_limits = {}
else:
    monthly_income = error.passMonthIncome()
    expenses = error.passData()

    months_to_archive = []
    for month_key in expenses["Expenses"]:
        if error.archivePrepDate(current_MonthYear, month_key):
            error.archiveData({month_key: expenses["Expenses"][month_key]})
            months_to_archive.append(month_key)

    for month in months_to_archive:
        if month in expenses["Expenses"]:
            del expenses["Expenses"][month]
    error.saveData(expenses)

    if current_MonthYear not in expenses["Expenses"]:
        expenses["Expenses"][current_MonthYear] = []
        error.saveData(expenses)

    remaining_balance = error.passRemBalance(current_MonthYear)
    total_expenses = error.passBudget(current_MonthYear)
    category_limits = error.budgetFlags(expenses, current_MonthYear)

style = ttk.Style()
style.theme_use('clam')
style.configure("Custom.TFrame", background=background_color)
style.configure("Custom.TLabel", background=background_color, font=base_font)
```

```
style.configure("Custom.TButton", background=primary_color, foreground="white",
font=base_font, padding=6)
style.map("Custom.TButton", background=[("active", accent_color)])
style.configure("Treeview", background=light_accent, foreground="black", rowheight=25,
                fieldbackground=light_accent, font=('Segoe UI', 10))
style.configure("Treeview.Heading", background=primary_color, foreground="white",
font=('Segoe UI', 11, 'bold'))


main_frame = ttk.Frame(root, padding=20, style="Custom.TFrame")
main_frame.pack(fill='both', expand=True)


income_frame = ttk.Frame(main_frame, style="Custom.TFrame")
income_frame.grid(row=0, column=0, sticky='ew', pady=10)
ttk.Label(income_frame, text="💰 Monthly Income:", style="Custom.TLabel").grid(row=0,
column=0, sticky='w')
income_entry = ttk.Entry(income_frame, font=base_font)
income_entry.grid(row=0, column=1, padx=5)

income_button = ttk.Button(income_frame, text="Set Income", style="Custom.TButton")
income_button.grid(row=0, column=2, padx=5)
totalExpenses_button = ttk.Button(income_frame, text="Total Spent", style="Custom.TButton")
totalExpenses_button.grid(row=0, column=3, padx=5)

archive_button = ttk.Button(income_frame, text="Past Expenses", style="Custom.TButton")
archive_button.grid(row=0, column=4, padx=5)


balance_label = ttk.Label(main_frame, text=f"Remaining Balance: PHP
{remaining_balance:.2f}", font=header_font, style="Custom.TLabel")
balance_label.grid(row=1, column=0, columnspan=3, pady=10)


def input_income():
    global monthly_income, remaining_balance, total_expenses
    try:
        income = float(income_entry.get())
        monthly_income = income
        remaining_balance = income
        balance_label.config(text=f"Remaining Balance: PHP {income:.2f}")


        income_entry.delete(0, tk.END)
```

```
        file.saveMonthIncome(income)

        # Switch button to "Change Income" mode
        income_button.config(text="Change Income", command=change_income)
    except ValueError:
        messagebox.showerror("Invalid Input:", "Please enter a valid numeric value for income.")
def change_income():
    global monthly_income, remaining_balance, total_expenses
    try:
        income = float(income_entry.get())
        monthly_income = income
        if total_expenses == 0.0:
            remaining_balance = income
        elif total_expenses <= income:
            remaining_balance = income - total_expenses
        else:
            messagebox.showerror("Balance Exceeded Monthly Income!", "Please delete an
expense.")
            remaining_balance = income - total_expenses

        balance_label.config(text=f"Remaining Balance: PHP {remaining_balance:.2f}")
        income_entry.delete(0, tk.END)
        file.saveMonthIncome(income)
    except ValueError:
        messagebox.showerror("Invalid Input:", "Please enter a valid numeric value for income.")

def showTotalSpent():
    messagebox.showinfo("Total Expenses", f"Your total spent for this month is
{total_expenses}")
def noMonthlyIncome():
    messagebox.showwarning("Total Expenses", f"Input Monthly Income first")

def pastExpenses():
    archiveDataDisplay = tk.Toplevel(root)
    archiveDataDisplay.title("Past Expenses")
    archiveDataDisplay.geometry("600x400")
    archiveDataDisplay.configure(bg='#f0f2f5')

    mini_frame = ttk.Frame(archiveDataDisplay, padding=20, style="Custom.TFrame")
    mini_frame.pack(fill='both', expand=True)



    mini_frame.columnconfigure(0, weight=1)
```

```
    mini_frame.rowconfigure(4, weight=1)

    archiveData = error.passArchive()
    prevMoth = list(archiveData.keys())

    ttk.Label(mini_frame, text="Select Month:", style="Custom.TLabel").grid(row=0, column=0,
sticky='w')
    prevMonth_combobox = ttk.Combobox(mini_frame, values=prevMoth, font=base_font,
state="readonly")
    prevMonth_combobox.grid(row=0, column=1, padx=5, pady=5)

    columns = ("ID", "Date", "Description", "Amount", "Category", "Payee", "Method")
    tree = ttk.Treeview(mini_frame, columns=columns, show="headings")
    for col in columns:
        tree.heading(col, text=col)
        tree.column(col, width=70, anchor='center')
    tree.grid(row=4, column=0, columnspan=3, pady=10, sticky='nsew')

    def on_month_selected(event):
        selectedMonth = prevMonth_combobox.get()
        tree.delete(*tree.get_children())
        for i, data in enumerate(archiveData[selectedMonth]):
            tree.insert('', 'end', values=(
                data["ID"],
                data["Date"],
                data["Description"],
                data["Amount"],
                data["Category"],
                data["Payee"],
                data["Method"]
            ))

    prevMonth_combobox.bind("<<ComboboxSelected>>", on_month_selected)
if monthly_income == 0 and remaining_balance == 0:
    income_button.config(text="Set Income", command=input_income)
    totalExpenses_button.config(state="normal", command=noMonthlyIncome)
else:
    income_button.config(text="Change Income", command=change_income)
    totalExpenses_button.config(state="normal", command=showTotalSpent)
archive_button.config(command=pastExpenses)



input_frame = ttk.Frame(main_frame, style="Custom.TFrame")
```

```
input_frame.grid(row=2, column=0, columnspan=3, sticky='ew', pady=10)

date_entry = ttk.Entry(input_frame, font=base_font)
desc_entry = ttk.Entry(input_frame, font=base_font)
amount_entry = ttk.Entry(input_frame, font=base_font)
category_combobox = ttk.Combobox(input_frame, values=["Food", "Transport", "Bills",
"Entertainment", "Savings", "Others"], font=base_font)
payee_entry = ttk.Entry(input_frame, font=base_font)
method_combobox = ttk.Combobox(input_frame, values=["Cash", "Credit", "Online", "Others"],
font=base_font)

ttk.Label(input_frame, text="📅 Date:", style="Custom.TLabel").grid(row=0, column=0,
sticky='w')
date_entry.grid(row=0, column=1, padx=5, pady=3)
ttk.Label(input_frame, text="📝 Description:", style="Custom.TLabel").grid(row=1, column=0,
sticky='w')
desc_entry.grid(row=1, column=1, padx=5, pady=3)
ttk.Label(input_frame, text="💸 Amount:", style="Custom.TLabel").grid(row=2, column=0,
sticky='w')
amount_entry.grid(row=2, column=1, padx=5, pady=3)
ttk.Label(input_frame, text="📂 Category:", style="Custom.TLabel").grid(row=0, column=2,
sticky='w')
category_combobox.grid(row=0, column=3, padx=5, pady=3)
ttk.Label(input_frame, text="👤 Payee:", style="Custom.TLabel").grid(row=1, column=2,
sticky='w')
payee_entry.grid(row=1, column=3, padx=5, pady=3)
ttk.Label(input_frame, text="💳 Method:", style="Custom.TLabel").grid(row=2, column=2,
sticky='w')
method_combobox.grid(row=2, column=3, padx=5, pady=3)


columns = ("ID", "Date", "Description", "Amount", "Category", "Payee", "Method")
tree = ttk.Treeview(main_frame, columns=columns, show="headings")

for col in columns:
    tree.heading(col, text=col)
    tree.column(col, width=130, anchor='center')
tree.grid(row=4, column=0, columnspan=3, pady=10, sticky='nsew')
i = 0

if file.isFileEmpty() == True:


    error.fillEmptyFile(current_MonthYear, monthly_income)
```

```
    expenses = error.passData()
else:
    months_to_archive = []

    for month_key in expenses["Expenses"]:
        if month_key == current_MonthYear:
            for i, data in enumerate(expenses["Expenses"][month_key]):
                tree.insert('', 'end', values=(
                    i + 1,
                    data["Date"],
                    data["Description"],
                    data["Amount"],
                    data["Category"],
                    data["Payee"],
                    data["Method"]
                ))
        elif error.archivePrepDate(current_MonthYear, month_key):
            error.archiveData({month_key:expenses["Expenses"][month_key]})
            months_to_archive.append(month_key)

    for month in months_to_archive:
        if month in expenses["Expenses"]:
            del expenses["Expenses"][month]
            error.saveData(expenses)

def input_expense():
    global expenses
    global remaining_balance

    amount = error.input_amount(amount_entry)
    if amount is None:
        return

    if amount > remaining_balance:
        messagebox.showerror("Not Enough Balance", "Not enough remaining balance to add this
expense!")
        return

    date = error.formatError(date_entry.get())
    if date is False:
        return


    category = error.input_category(category_combobox)
```

```
    if category is None:
        return

    method = error.input_method(method_combobox)
    if method is None:
        return

    description = error.input_description(desc_entry)
    if description is None:
        return

    payee = error.input_payee(payee_entry)
    if payee is None:
        return

    category_limits.setdefault(category, 0)
    category_limits[category] += amount

    if category_limits[category] > monthly_income * 0.5:
        messagebox.showwarning("Overspending", f"Overspending in category '{category}'.")

    dateInputted = error.cmprdates(date, current_MonthYear)

    if dateInputted is None or (dateInputted is False and datetime.datetime.strptime(date,
"%m/%d/%Y") < datetime.datetime.strptime(current_MonthYear, "%B %Y")):
        return

    expense_data = {
        "Date": date,
        "Description": description,
        "Amount": amount,
        "Category": category,
        "Payee": payee,
        "Method": method
    }

    if dateInputted == True:
        expense_id = logic.addCurrentMonthExpense(expenses, current_MonthYear)
        remaining_balance -= amount
        balance_label.config(text=f"Remaining Balance: PHP {remaining_balance:.2f}")

        expense_data["ID"] = expense_id

        tree.insert('', 'end', values=(
```

```
            expense_id,
            date,
            description,
            f"{amount:.2f}",
            category,
            payee,
            method
        ))

        expenses["Expenses"][current_MonthYear].append(expense_data)
        error.saveData(expenses)

    else:
        next_MonthYear = error.datetoKey(date)
        expense_data["ID"] = logic.addNextMonthExpense(expenses, current_MonthYear,
next_MonthYear)
        expenses["Expenses"][next_MonthYear].append(expense_data)
        error.saveData(expenses)

    date_entry.delete(0, tk.END)
    desc_entry.delete(0, tk.END)
    amount_entry.delete(0, tk.END)
    category_combobox.delete(0, tk.END)
    payee_entry.delete(0, tk.END)
    method_combobox.delete(0, tk.END)


def delete_selected():
        global expenses
        global remaining_balance

        selected = tree.selection()
        if not selected:
            messagebox.showwarning("No Selection", "Please select a row to delete.")
            return

        if current_MonthYear not in expenses["Expenses"]:
            messagebox.showerror("Error", f"No expenses found for {current_MonthYear}.")
            return

        for item in selected:
            values = tree.item(item, 'values')
            amount = float(values[3])
            category = values[4]
```

```
        item_id = int(values[0])

        for i, exp in enumerate(expenses["Expenses"][current_MonthYear]):
            if exp["ID"] == item_id:
                expenses["Expenses"][current_MonthYear].pop(i)
                break

        remaining_balance += amount
        balance_label.config(text=f"Remaining Balance: PHP {remaining_balance:.2f}")

        if category in category_limits:
            category_limits[category] -= amount
            if category_limits[category] <= 0:
                del category_limits[category]
        tree.delete(item)

    tree.delete(*tree.get_children())
    for i, data in enumerate(expenses["Expenses"][current_MonthYear], start=1):
        data["ID"] = i
        tree.insert('', 'end', values=(
            i,
            data["Date"],
            data["Description"],
            f"{data['Amount']:.2f}",
            data["Category"],
            data["Payee"],
            data["Method"]
        ))

    error.saveData(expenses)

ttk.Button(main_frame, text="➕ Add Expense", style="Custom.TButton",
command=input_expense).grid(row=3, column=0, columnspan=3, pady=10)
ttk.Button(main_frame, text="🗑️ Delete Selected", style="Custom.TButton",
command=delete_selected).grid(row=5, column=0, columnspan=3, pady=10)

main_frame.grid_rowconfigure(4, weight=1)
main_frame.grid_columnconfigure(0, weight=1)
main_frame.grid_columnconfigure(1, weight=1)
main_frame.grid_columnconfigure(2, weight=1)

root.mainloop()
```

(from *“import datahandling as file”)*

```
import json
import logic as logic
import errorhandling as error

def isFileEmpty():
    with open('expense data.json') as f:
        content = f.read().strip()
    if not content:
        return True
    else:
        return False

def saveMonthIncome(income):
    with open('expense data.json') as f:
        data = json.load(f)
    data["Monthly Income"] = income
    with open('expense data.json', 'w') as f:
        json.dump(data, f, indent=4)
```

(from *"import datehandling as datehandle")*

```
import datetime

def passDate():
    current_date = datetime.datetime.now()
    current_MonthYear = current_date.strftime("%B %Y")
    return current_MonthYear
```

(from *"import errorhandling as error")*

```
import datetime
from tkinter import messagebox
import json

#Error-Logic Handling
def calculate_remaining_balance(data, currentDate):
    remBalance = 0
    try:
        expenses = data["Expenses"].get(currentDate, [])
        for entry in expenses:
            remBalance += entry.get("Amount", 0)
        return data.get("Monthly Income", 0) - remBalance


    except Exception as e:
```

```
        messagebox.showerror("Calculation Error", f"ERROR: {e}")
        return None

def budgetFlags(expenses, current_MonthYear):
    category_limits = {}
    try:
        for limit in expenses["Expenses"][current_MonthYear]:
            category = limit["Category"]
            amount = limit["Amount"]
            if category in category_limits:
                category_limits[category] += amount
            else:
                category_limits[category] = amount
        return category_limits
    except Exception as e:
        messagebox.showerror("Calculation Error", f"ERROR: {e}")
        return None

def calculate_total_spent(data, currentDate):
    try:
        total = 0
        for i in range(len(data["Expenses"][currentDate])):
            total += data["Expenses"][currentDate][i]["Amount"]
        return total
    except Exception as e:
        messagebox.showerror("Calculation Error", f"ERROR: {e}")
        return None

#Error-File Handling

def passArchive():
    try:
        with open('archivedexpenses.json') as f:
            data = json.load(f)
        return data
    except (FileNotFoundError, json.JSONDecodeError) as e:
        messagebox.showerror("File Error", f"ERROR: archivedexpenses.json cannot be loaded:
{e}")
        return {}

def archiveData(data):


    try:
```

```
        with open('archivedexpenses.json', 'w') as f:
            json.dump(data, f, indent=4)
    except TypeError as e:
        messagebox.showerror("Save Error", f"ERROR: Saving archive data failed: {e}")
        return {}

def passData():
    try:
        with open('expense data.json') as f:
            data = json.load(f)
        return data
    except (FileNotFoundError, json.JSONDecodeError) as e:
        messagebox.showerror("File Error", f"ERROR: expense expense data.json cannot be loaded:
{e}")
        return None

def fillEmptyFile(currentDate, income):
    try:
        with open('expense data.json', 'w') as f:
            data = {
                "Monthly Income": income,
                "Expenses": {
                    currentDate: []
                }
            }
            json.dump(data, f, indent=4)
    except Exception as e:
        messagebox.showerror("File Error",f"Error creating expense data file: {e}")

def passMonthIncome():
    try:
        with open('expense data.json') as f:
            data = json.load(f)
        return data['Monthly Income']
    except (FileNotFoundError, json.JSONDecodeError, KeyError) as e:
        messagebox.showerror("Data Error", f"Error retrieving monthly income: {e}")
        return None

def passRemBalance(currentDate):
    try:
        with open('expense data.json') as f:


            data = json.load(f)
```

```
        remBalance = calculate_remaining_balance(data, currentDate)
        return remBalance
    except (FileNotFoundError, json.JSONDecodeError, KeyError) as e:
        messagebox.showerror("Data Error", f"Error retrieving remaining balance: {e}")
        return None

def passBudget(currentDate):
    try:
        total = 0
        with open('expense data.json') as f:
            data = json.load(f)
        total = calculate_total_spent(data, currentDate)
        return total
    except (FileNotFoundError, json.JSONDecodeError, KeyError) as e:
        messagebox.showerror("Data Error", f"Error retrieving total expenses: {e}")
        return None

def saveData(data):
    try:
        with open('expense data.json', 'w') as f:
            json.dump(data, f, indent=4)
    except TypeError as e:
        messagebox.showerror("Save Error", f"ERROR: Saving expense data failed: {e}")

#Error-Date Handling
def archivePrepDate(currentdate, dates):
    format = "%B %Y"
    current_dt = datetime.datetime.strptime(currentdate, format)
    target_dt = datetime.datetime.strptime(dates, format)
    return target_dt < current_dt

def formatError(userdate):
    diff_formats = [
        "%d %B, %Y",
        "%B %d, %Y",
        "%m/%d/%Y",
        "%d/%m/%Y",
        "%Y-%m-%d",
        "%Y-%m-%d",
        "%m-%d-%Y",
        "%B-%d-%Y",


    ]
```

```
    for fmt in diff_formats:
        try:
            transformed_date = datetime.datetime.strptime(userdate, fmt)
            return transformed_date.strftime("%m/%d/%Y")
        except ValueError:
            continue
    if not userdate:
        messagebox.showerror("Missing Date", f"Please enter date.")
        return False
    messagebox.showerror("Date Error", f"ERROR: Unable to read the date '{userdate}' in any
supported format.")
    return False

def cmprdates(input_date, currentdate):
    input_dt = datetime.datetime.strptime(input_date, "%m/%d/%Y")
    current_dt = datetime.datetime.strptime(currentdate, "%B %Y")
    input_month = input_dt.replace(day=1)
    current_month = current_dt.replace(day=1)

    if input_month == current_month:
        return True
    elif input_month > current_month:
        messagebox.showinfo("Scheduled for Future Month", "Your expense was scheduled for a
future month.")
        return False
    else:
        messagebox.showerror("Invalid Date", "You cannot schedule an expense for a past
month.")
        return False

def datetoKey(date):
    # Try parsing the date using different formats
    date_formats = [
        "%d %B, %Y",
        "%B %d, %Y",
        "%m/%d/%Y",
        "%d/%m/%Y",
        "%Y-%m-%d",
        "%Y-%m-%d",
        "%m-%d-%Y",
        "%B-%d-%Y",
    ]

    for fmt in date_formats:
```

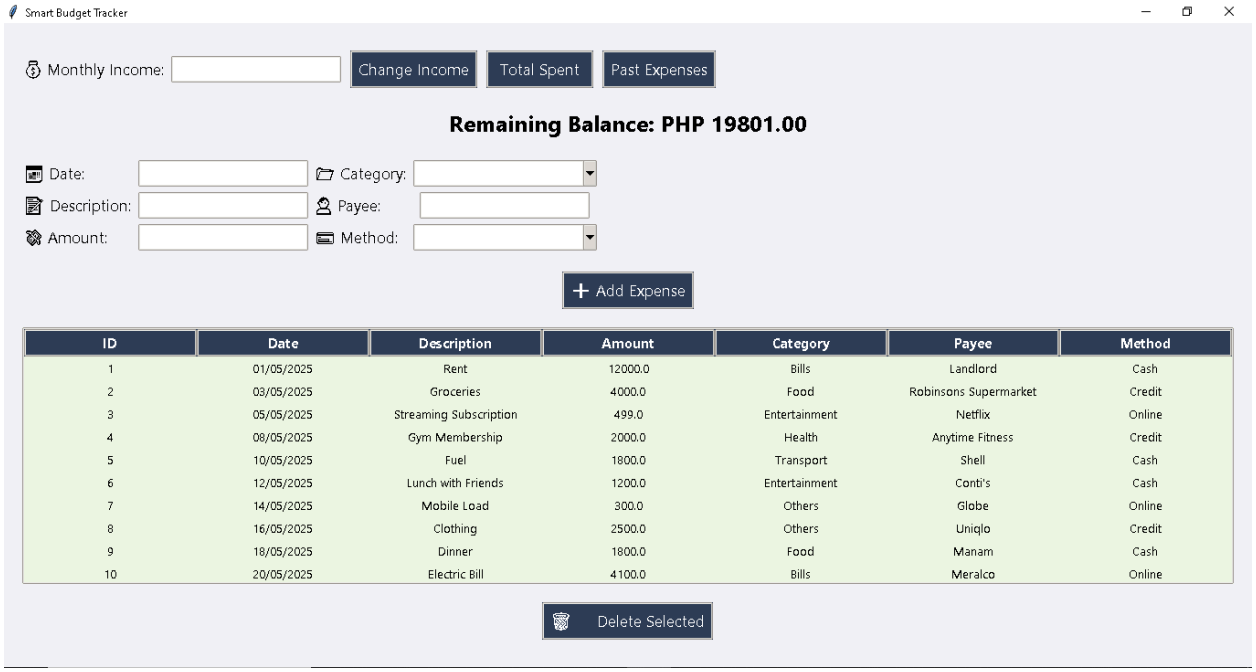
```
        try:
            nextdate = datetime.datetime.strptime(date, fmt)
            return nextdate.strftime("%B %Y")
        except ValueError:
            continue
    messagebox.showerror("Date Format Error", f"Date format of '{date}' is not valid.")

#Others
def input_description(desc_entry):
    description = desc_entry.get()

    if not description:
        messagebox.showwarning("Missing Description", "Please fill out the description field.")
        return None
    return description

def input_payee(payee_entry):
    payee = payee_entry.get()

    if not payee:
        messagebox.showwarning("Missing Payee", "Please fill out the payee field.")
        return None
    return payee

def input_category(category_combobox):
    category = category_combobox.get()

    if not category:
        messagebox.showwarning("Missing Category", "Please select a category.")
        return None
    return category

def input_method(method_combobox):
    method = method_combobox.get()

    if not method:
        messagebox.showwarning("Missing Method", "Please select a payment method.")
        return None
    return method

def input_amount(amount_entry):


    raw = amount_entry.get().strip()
```

```
    if not raw:
        messagebox.showerror("Missing Input", "Amount field is required.")
        return None
    try:
        amount = float(raw)
        if amount < 0:
            messagebox.showerror("Invalid Amount", "Amount cannot be negative.")
            return None
        return amount
    except ValueError:
        messagebox.showerror("Invalid Input", "Please enter a valid number for amount.")
        return None
```

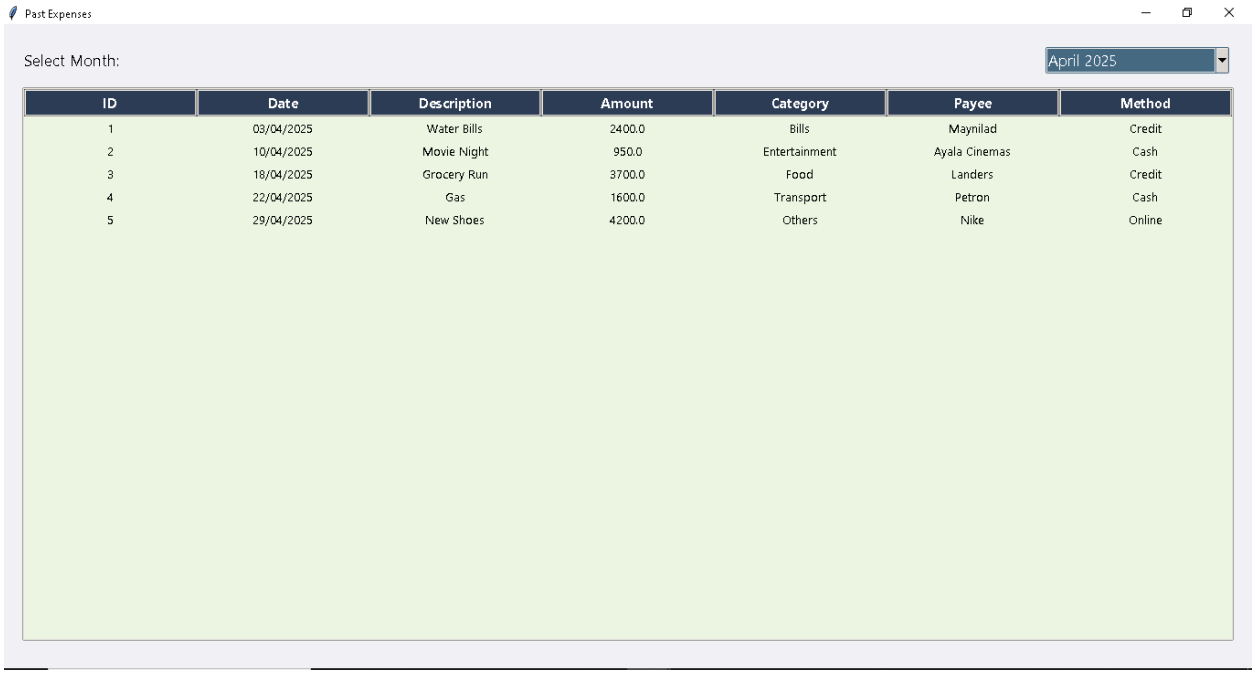
(from *"import logic as logic")*

```
def addCurrentMonthExpense(expenses, current_MonthYear):
    existing = expenses["Expenses"].get(current_MonthYear, [])
    start_id = max([item["ID"] for item in existing], default=0)
    expense_id = start_id + 1
    return expense_id

def addNextMonthExpense(expenses, current_MonthYear, next_MonthYear):
    if next_MonthYear not in expenses["Expenses"]:
        expenses["Expenses"][next_MonthYear] = []

    if not expenses["Expenses"][next_MonthYear]:
        expense_id = 1
    else:
        last_id = expenses["Expenses"][next_MonthYear][-1]["ID"]
        expense_id = last_id + 1
    return expense_id
```

SYSTEM LAYOUT/DESIGN



ID	Date	Description	Amount	Category	Payee	Method
1	01/05/2025	Rent	12000.0	Bills	Landlord	Cash
2	03/05/2025	Groceries	4000.0	Food	Robinsons Supermarket	Credit
3	05/05/2025	Streaming Subscription	499.0	Entertainment	Netflix	Online
4	08/05/2025	Gym Membership	2000.0	Health	Anytime Fitness	Credit
5	10/05/2025	Fuel	1800.0	Transport	Shell	Cash
6	12/05/2025	Lunch with Friends	1200.0	Entertainment	Conti's	Cash
7	14/05/2025	Mobile Load	300.0	Others	Globe	Online
8	16/05/2025	Clothing	2500.0	Others	Uniqlo	Credit
9	18/05/2025	Dinner	1800.0	Food	Manam	Cash
10	20/05/2025	Electric Bill	4100.0	Bills	Meralco	Online



ID	Date	Description	Amount	Category	Payee	Method
1	03/04/2025	Water Bills	2400.0	Bills	Maynilad	Credit
2	10/04/2025	Movie Night	950.0	Entertainment	Ayala Cinemas	Cash
3	18/04/2025	Grocery Run	3700.0	Food	Landers	Credit
4	22/04/2025	Gas	1600.0	Transport	Petron	Cash
5	29/04/2025	New Shoes	4200.0	Others	Nike	Online

CURRICULUM VITAE

JULIEN JAMILE P. AVILA

Contact

4 Mataba, Cubao, Quezon City, Metro Manila

+63 910 439 5854

shs.avilajulienjamile@gmail.com

Personal Details

Age: 19
Birthday: December 10, 2005
Birthplace: Toronto, Canada
Qualities: Approachable, Innovative, Professional, Diligent, Honest

Skills

Leadership | Management
Logical Thinking | Teamwork
Problem Solving | Consistency

Summary Statement

As a determined Computer Science student, I'm always eager to seize opportunities that help me grow both personally and professionally. My drive to push beyond my limits keeps me motivated to work hard and continuously improve. With a strong commitment to learning and developing my skills, I desire not only to gain valuable experiences but also to contribute meaningfully to the goals and visions of organizations I associate with in the incoming future.

Education

College
2024 - Present
Pamantasan ng Lungsod ng Maynila
Bachelor of Science in Computer Science

Senior High School
2022 - 2024
Adamson University
Science, Technology,Engineering and Mathematics, Specializing in Technology (STECH)

Junior High School
2018 - 2022
Makati High School

Technical Skills

Basic Video Editing Skills
Filmora, Capcut, Priemere Pro, and After Effects
Basic Graphics Designing Skills
Canva and Photoshop
Basic Programming Skills
HTML,CSS, and JavaScript; C# (Windows Form); C/C++; Python; Arduino (C++)

Achievements

- *Graduated Grade 12* ***With Honors***
- *Special Participation Award* from the **International Competition on Internet of Things 2023**
- **Best Qualitative Research Proposal** - STECH
- ***With High Honors*** during the 11th Grade (2023)
- ***With Honors*** during the 10th Grade (2022)
- *3rd Place* in **Beyond Poetry Video-Making Contest** in the **National Reading Month 2021**
- *First Place* in **Science Concept Quick Share** in the **2021 Virtual Science Month Celebration**
- *With Honors* during the 9th Grade(2021)

Professional References

Claire A. Garcia
Chemistry Instructor, Adamson University Senior High School
claire.garcia@adamson.edu.ph
(+63)9357363282

Rico T. Teodoro
Faculty Member, Adamson University Senior High School
rico.teodoro@adamson.edu.ph

Mary Abigail J. Aniban
Faculty Member, Adamson University Senior High School
(+63)9771052068

LOREJANE M.
DELA CRUZ

Contact

Salitran 2, Dasmariñas City, Cavite

+63 915 883 6593

lorejane.millan.delacruz@gmail.com

Personal Details

Age: 19
Birthday: January 4, 2006
Birthplace: Dasmariñas City, Cavite
Qualities:

Skills

Creativity
Adaptability
Communication
Problem Solving

Summary Statement

An aspiring Computer Science student with a solid foundation in programming and a strong passion for solving real-world problems through technology. Demonstrates a willingness to learn and grow continuously in the field to make meaningful contributions."

Education

College
2024 - Present
Pamantasan ng Lungsod ng Maynila
Bachelor of Science in Computer Science

Senior High School
2022 - 2024
Emilio Aguinaldo College - Cavite
Science, Technology, Engineering, and Mathematics (STEM)

Junior High School
2018 - 2022
Dasmariñas Integrated High School

Technical Skills

Microsoft Office Skills
Word, Powerpoint, Excel

Basic Graphics Designing Skills
Canva, Figma, Picsart and Photoshop

Basic Programming Skills
HTML, CSS, C, and Python

Achievements

- *Graduated **With Honors** Grade 12 (2024)*
- ***With Honors*** during the 11th Grade (2023)
- ***With high Honors*** during the 10th Grade (2022)

Professional References

Dr. George E. Gerpheide
Client, Freelancing
george.gerpheide@gmail.com

Bren Alfred Espana
Faculty Member, Pamantasan ng Lungso ng Maynila
baespana@*plm.edu.ph*

Richard Regala
Faculty Member, Pamantasan ng Lungso ng Maynila
richardregala@plm.edu.ph

GABRIELLE RUTH V. GUEVARRA

Contact

1D Greenview Subd., Pamplona Tres Las Pinas City

+63 976 340 1354

gbbyrvg@gmail.com

Personal Details

Age: 20
Birthday: December 3, 2004
Birthplace: Las Pinas City
Qualities: Organized,
Empathy, Trustworthy, Reliable, Honest

Skills

Time Management | Responsible | Task-Oriented | Computer Literate | Photography | Video Editing

Summary Statement

Dedicated and goal-driven Computer Science major who is passionate about technology, creativity, and problem-solving. Brings hands-on experience in video editing, computer applications, and foundational programming. Recognized for academic excellence, strong interpersonal skills, and being a reliable team player with a responsible, time-conscious work ethic, dedicated to producing high-quality results in every task.

Education

College
2024 - Present
Pamantasan ng Lungsod ng Maynila
Bachelor of Science in Computer Science

Senior High School
2022 - 2024
Las Pinas City National Science High School
Science, Technology, Engineering and Mathematics, (STEM)

Junior High School
2018 - 2022
Las Pinas City National Science High School

Technical Skills

Basic Video Editing Skills
Filmora, Capcut, and DaVinci Resolve

Basic Computer Skills
Ms Word, Ms Excel, Ms Powerpoint, Google Docs, Google Sheets, Google Forms

Basic Programming Skills
HTML, CSS, C, JavaScript, and Python

Achievements

- *Graduated Grade 12* ***With Honors***
- ***Outstanding Performance in Work Immersion*** (2024)
- **Conduct Awardee** (2024)
- *Consistent* ***With High Honors*** *during Grades 9th to 11th (2020-2023)*
- *Consistent* ***With Honors*** *during Grades 7th to 8th (2018-2020)*
- ***Layout, Design, Photos, and Illustrations Member*** *for The LPSci Standard Press (2023-2024)*
- **Red Cross Youth Member** (2018-2019)

Professional References

Dianne Gasgonia
Electronics and Communications Engineering Student, University of the Philippines Diliman
(+63)917 552 1009

Jarred Matthew Pascual
Biology *Student, University of the Philippines Manila*
(+63)916 634 0449

Mark Raven Pastrana
Mechanical *Engineering Student, Pamantasan ng Lungsod ng Maynila*
markpastrana012306@gmail.com
(+63)906 467 0315

RUSSEL PAOLO C. JANDOC

Contact

231, Balagtas St. Tondo Manila

+63 999 169 8609

russelpaolojandoc@gmail.com

Personal Details

Age: 19
Birthday: August 31, 2005
Birthplace: Manila
Qualities: Curiosity, Generosity
Empathy, Trustworthy, Reliable, Honest, Learning

Skills

Time Management | Responsible | Task-Oriented | Computer Literate | Teamwork | Coaching | Collaboration

Summary Statement

Motivated and enthusiastic Computer Science student with a strong desire to learn and grow in the tech industry. Equipped with foundational knowledge in programming and problem-solving, and eager to explore emerging technologies. Passionate about using technology to create practical solutions and improve everyday experiences. Known for being committed, collaborative, and dedicated to continuous improvement through hands-on experience.

Education

College
2024 - Present
Pamantasan ng Lungsod ng Maynila
Bachelor of Science in Computer Science

Senior High School
2022 - 2024
National Teachers College (NTC)
Science, Technology, Engineering and Mathematics, (STEM)

Junior High School
2018 - 2022
National Teachers College

Technical Skills

Basic Graphics Designing Skills
Canva, Picsart

Basic Computer Skills
Ms Word, Ms Excel, Ms Powerpoint, Google Docs, Google Sheets, Google Forms

Basic Programming Skills
HTML, CSS, C, JavaScript, and Python

Achievements

- *Graduated Grade 11 and Grade 12* ***With High Honors*** *(2022 - 2024)*
- *Consistent* ***With High Honors*** *during Junior High (2018-2022)*

Professional References

Marijane Jandoc
VPAA, Pamantasan ng Lungsod ng Maynila
mcjandoc@plm.edu.com
(+63)9991698609

Geneva Aira B. Trinidad
Physical Therapy Student,
College of Pamantasan Ng Lungsod Ng Maynila
gabtrinidad2024@plm.edu.ph
(+63)9708236997

Benedict Christian S. Bustillos
Architecture Student, National University-Manila
Benchbustillos8@gmail.com
(+63)9943503482

SCHEANZ YONCE MARTIN

Contact

1954, Leveriza Street Pasay City

+63 9284200316

scheanz3@gmail.com

Personal Details

Age: 19
Birthday: August 26, 2005
Birthplace: Manila, Philippines
Qualities: Passionate, Honest, Resilient, and Self-Awareness

Skills

Critical Thinking
Adaptability
Work Ethic
Time Management
Teamwork
Consistency

Summary Statement

Aspiring computer science major with a goal to innovate solutions to real-world problems that will bridge technology and real-world needs. Skilled in software development, collaborative works, and data management. Committed to continuous learning while building projects, improving technologies, and creating impactful outcomes. Passionate about using technology as a tool for problem-solving, efficiency, and social good.

Education

College
2024 - Present
Pamantasan ng Lungsod ng Maynila
Bachelor of Science in Computer Science

Senior High School
2022 - 2024
Arellano University, Apolinario Mabini Campus
Information Communication Technology (ICT)

Junior High School
2018 - 2022
Malate Catholic School (MCS)

Technical Skills

Basic Programming Skills
HTML, JS, CSS, C, PYTHON, & JAVA
Basic Graphics Designing Skills
Canva, Picsart, and Photoshop
Microsoft Office Skills
Word, Powerpoint, Excel

Achievements

- Consistent **with High Honors** in Senior High (2022 - 2024)
- Consistent **with High Honors** in Junior High (2018 - 2022)
- ICT-B **RANK 1** in grade 12 (2023-2024)
- ICT-B **RANK 1** in grade 11 (2022-2023)
- ICT **Board Executive** member (2022 - 2024)
- **Top 8 in Senior High School at Arellano University Apolinario Mabini Campus**

Professional References

John Vincent Sadie
ICT Faculty Member,
Arellano University Senior High School
johnvsadie@gmail.com

Marjorie Tilles
ICT Faculty Member,
PCU Senior High School
mtilles@pcushs.edu.ph

JAN ZYANNE B. SEBASTIAN

Contact

1842 Biak Na Bato Street, Tondo Manila

+63 927 664 8266

zyn620@gmail.com

Personal Details

Age: 19
Birthday: June 20, 2005
Birthplace: Manila, Philippines
Qualities: Honest, Confident, Reliable, Willing to learn

Skills

Creativity | Time Management | Communication | Adaptability | Problem Solving | Consistency

Summary Statement

Motivated and goal-oriented Computer Science student with a basic foundation in software development, communication, and interdisciplinary collaboration. Experienced in content creation, student governance, and community service, with a passion for using technology to address real-world challenges. Demonstrated ability to manage projects, lead organizations, and communicate effectively across diverse platforms.

Education

College
2024 - Present
Pamantasan ng Lungsod ng Maynila
Bachelor of Science in Computer Science

Senior High School
2022 - 2024
FEU High School
Science, Technology,Engineering and Mathematics (STEM)

Junior High School
2018 - 2022
Espiritu Santo Parochial School

Technical Skills

Basic Programming Skills
HTML, CSS, Java Script, C, and Python

Basic Graphics Designing Skills
Canva, Picsart, and Photoshop

Microsoft Office Skills
Word, Powerpoint, Excel

Achievements

- Consistent **with Honors** in Senior High (2022 - 2024)
- Consistent **with High Honors** in Junior High (2018 - 2022)
- **Senior Creative Researcher** for FEU High School's Broadcasting System (2022 - 2024)
- **Marketing Staff** for FEU High School's Broadcasting System (2022 - 2023)
- **Technical Staff** for FEU High School's Young Leaders for the Environment (2023 - 2024)
- **Vice President** of ESPS' Science Club MaSCIya (2019 -2021)

Professional References

Wayne Francis Regis
STEM Faculty Member,
FEU High School
wfregis@feuhighschool.edu.ph

Mitzi Rose Godmalin
STEM Faculty Member,
FEU High School
mgodmalin@feuhighschool.edu.ph

Heine Rodulfa
STEM Faculty Member,
FEU High School
hrodulfa@feuhighschool.edu.ph